I/O Software Layers

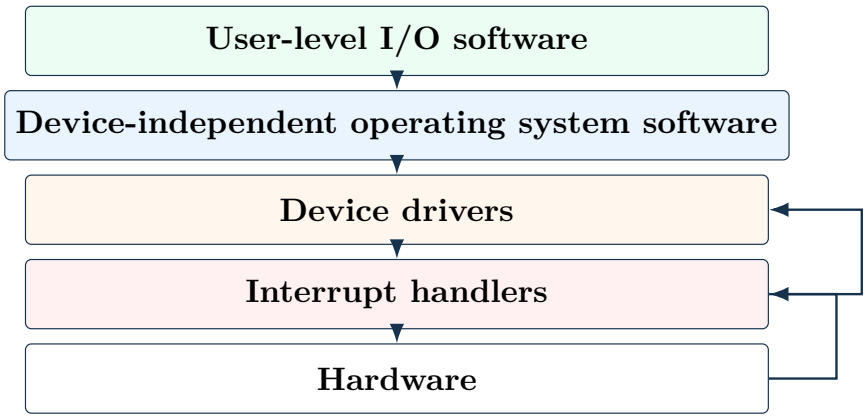
1 Basic Idea

Definition

I/O software layers are the operating-system layers that organize I/O work from user-level requests down to hardware operations and interrupt handling.

- Each layer has a specific function.
- Each layer communicates with adjacent layers through a defined interface.
- The exact design differs between operating systems.
- The main goal is to hide low-level device complexity from most of the OS and from user programs.

2 Layered Structure



Layer	Main responsibility
User-level I/O software	Provides libraries, formatting routines, and user-visible I/O interfaces.
Device-independent OS software	Provides common naming, protection, buffering, allocation, and error handling.
Device drivers	Contains device-specific code and knows controller commands and registers.
Interrupt handlers	Respond to hardware interrupts and wake blocked driver code.
Hardware	Performs the actual data transfer and raises interrupts.

3 Why Use Layers

- Layers separate general I/O policy from device-specific mechanisms.

- User programs do not need to know controller registers or interrupt details.
- Most of the operating system can call uniform I/O interfaces.
- Device-specific complexity is mostly isolated inside drivers.
- Interrupt details are hidden as deeply as possible.

Main Benefit

Layering makes I/O easier to manage because only the lowest layers deal directly with hardware timing, registers, and interrupts.

4 Interrupt Handlers

Role

Interrupt handlers are low-level routines that run when hardware interrupts the CPU.

Interrupts are needed for most real I/O, but they are difficult to handle cleanly. The OS tries to hide interrupts deep inside the kernel. Ideally, higher OS layers see a completed I/O operation, not raw interrupt events.

5 Blocking the Driver

Driver Blocking Model

A driver starts an I/O operation and then blocks until the interrupt reports that the operation has completed.

The driver may block on a semaphore, condition variable, or message. This model works especially well when drivers are kernel processes, because a kernel-process driver has its own state, stack, and program counter.

Blocking method	Matching unblock action
Semaphore	The interrupt handler performs an up operation.
Condition variable	The interrupt handler signals the condition variable.
Message passing	The interrupt handler sends a message to the blocked driver.

Effect of the Interrupt

The interrupt handler performs the low-level interrupt work and then unblocks the driver that was waiting for I/O completion.

6 Interrupt Processing Steps

Not Just IRET

Handling an interrupt is not only acknowledging the interrupt and returning. The OS may need to save state, set up memory context, run a handler, schedule, and restore a process.

1. Save registers and processor status if hardware has not already done so.
2. Set up a context and stack for the interrupt-service procedure.
3. Acknowledge the interrupt controller or reenable interrupts.
4. Copy saved registers into the process table if needed.
5. Run the interrupt-service procedure and read controller status.
6. Choose which process should run next.
7. Set up the MMU and TLB context for that process if needed.
8. Load the selected process's registers and processor status.
9. Start running the selected process.

7 Why Interrupt Processing Is Expensive

- Register state may need to be saved and restored.
- The process table may need to be updated.
- The scheduler may need to choose a new process.
- Virtual memory systems may require MMU and page-table changes.
- TLB entries may need to be changed or reloaded.
- CPU cache behavior may be affected when switching between user and kernel mode.

Final Point

Interrupt handlers are placed at the bottom of the I/O software stack because interrupts are hardware-specific, expensive, and best hidden from higher-level OS code.